הפקולטה להנדסה
ע"ש אלכסנדר קופקין
אוניברסיטת בר־אילן

אוניברסיטת
בר־אילן
Bar-Ilan University

Final project in computer engineering on the subject

taoFPGA: Architecture, Implementation, and Evaluation of a Quantized Transformer Processing Core on Edge FPGA

taoFPGA: ארכיטקטורה, מימוש והערכת ביצועים של מאיץ חומרה מקוונטט לטרנספורמרים על גבי FPGA במכשירי קצה

Authors:

Eliran Turgeman, 316372002

Shay Rask, 314951658

Track: Computer Engineering — Hardware and Chip Design

Academic supervisor: Dr. Leonid Yavits (Leonid.yavits@biu.ac.il)

Project mentor: David Freud (david.freud@mail.huji.ac.il)

Date of submission: September 22, 2026

Submitted as part of the undergraduate requirements

Faculty of Engineering, Bar-Ilan University

Project number: 309

ABSTRACT

Transformer models are increasingly deployed on power- and resource-constrained edge devices, where their quadratic attention cost and large dense matrix multiplications strain both compute and memory bandwidth. This report presents taoFPGA, a quantized INT8 transformer processing core built around a 16×16 weight-stationary systolic array fused with dedicated fixed-point Softmax and GELU engines, verified end to end from RTL simulation through physical signoff and real hardware bring-up on a Xilinx Zynq-7020 (PYNQ-Z2). We detail a tolerance-bounded verification methodology carried consistently from SystemVerilog testbenches through a Python/NumPy golden model validated against physical silicon, a synthesis-level DSP-inference defect whose correction reduced LUT utilization by 49.3% and total on-chip power by 3.1%, and a full-scale hardware validation run matching simulation to zero error across 32,000 output elements. We further report a kernel-level benchmark against a real, numerically validated ViT-Tiny software baseline, achieving up to 97.73× measured speedup (70.02× and 97.73× across the two benchmarked kernel shapes) over the board's ARM Cortex-A9, corresponding to a total energy efficiency of 2.25 GOPS/W (2.47 GOPS/W on dynamic power alone) at peak throughput -- while explicitly analyzing why the accelerator's fixed pipeline does not structurally correspond to any single Vision Transformer operation, a distinction we treat as a methodological contribution in its own right. We close with a quantified account of the design's remaining hardware constraints, principally a 64KB single-transfer DMA ceiling, and the Scatter-Gather DMA redesign identified as its prerequisite fix.

ACKNOWLEDGMENTS

We would like to thank our academic supervisor, Dr. Leonid Yavits, and our project mentor, David Freud, for their guidance throughout this project.

Table of Contents

Abstract 3
Acknowledgments 3
Preliminaries 6
A. Transformer Self-Attention 6
B. Fixed-Point (INT8) Quantization 6
C. Systolic Arrays for Matrix Multiplication 6
I. Introduction & Motivation 6
A. The Edge AI Compute Dilemma 6
B. Design Intent & Methodology 6
C. Key Contributions & Paper Outline 7
II. Core Hardware Architecture & Mathematical Formulation 7

A. Systolic Matrix Multiplication Core 7
B. Fixed-Point Non-Linear Arithmetic Engines 8
C. Numerical Quantization Strategy 9
III. RTL Verification & Behavioral Simulation (Cadence Xcelium) 9
A. Testbench Architecture & Golden Reference Model 9
B. Simulation Results & Numerical Precision Verification 9
C. Pre-Silicon Logic Debugging 9
IV. SoC Integration & Hardware Platform Design (PYNQ-Z2 / Zynq-7020) 9
A. SoC Architecture 9
B. Physical Adaptations for Board Deployment 10
C. Software Infrastructure for Hardware Bring-Up 11
V. Physical Implementation, P&R, and Sign-Off Optimizations 11
A. Baseline Physical Sign-Off 11
B. The DSP Inference Gap & Attribute Resolution 11
C. Physical Congestion vs. Timing Trade-Off 11
D. Bitstream & Hardware Platform Sign-Off 12
VI. Experimental Evaluation & Hardware Bring-Up 12
A. Benchmarking Methodology 12
B. Performance Metrics & Acceleration 12
C. Physical Silicon Verification 13
D. Workload Characterization vs. ViT Networks 13
E. Hardware Bring-Up Observations & Edge Cases 13
F. Comparative Analysis with Prior FPGA Transformer Accelerators 14
VII. Engineering Discussion, Bottlenecks & Future Work 14
A. DMA Buffer Boundary Constraints 14
B. Serial Softmax Throughput Bottleneck 14
C. Architectural Roadmap 14
VIII. Conclusion 15
References 15
Appendix A: Engineering Problems Encountered and Their Solutions 16
A.1 Elaboration-Time Memory Blowup From `generate`-Unrolled Test Arrays 16
A.2 A Silent Compile Bug Hidden by an Untested Code Path 16
A.3 A Documented, Deliberately-Deferred Bug That Was Still a Real Bug 17
A.4 Six Silent Parameter Mismatches Between Verified RTL and an Adopted Integration Script 17

A.5 Async-Reset DRC Violations: Chasing One Root Cause Through Twelve Files 17
A.6 Discovering the Full Pipeline Had Never Actually Been AXI-Wrapped 18
A.7 A Synthesis Attribute That Silently Did Nothing 18
A.8 Same Chip, Different Board: the PYNQ-Z1 to PYNQ-Z2 Hardware Audit 19
A.9 A Comprehensively Stale Software Driver, Found and Rebuilt From the Real Register Map 19
A.10 A Real, Reproducible Hardware/Software Mismatch, Diagnosed Methodically 20
A.11 The Real Root Cause: a Tiny-Shape Edge Case, Not a Design Flaw 20
A.12 Refusing to Report a Speedup the Pipeline Doesn't Actually Support 20
A.13 A 64KB DMA Ceiling, and an Unsafe-Looking Workaround 21
Appendix B: General Engineering Principles 22

List of Figures

Fig. 1. One PE's internal datapath (stationary weight, INT8 multiply, systolic propagation). 8
Fig. 2. Softmax_control's 3-pass FSM and its per-pass equations. 8
Fig. 3. Illustrative AXI4-Stream handshake and 3-pass Softmax timing structure. 9
Fig. 4. Complete top-level SoC architecture. 10
Fig. 5. Latency, CPU vs. hardware, both benchmarked shapes. 12
Fig. 6. Throughput (GOP/s), CPU vs. hardware, both shapes. 12
Fig. 7. Measured hardware kernel speedup over CPU. 13
Fig. 8. Real ViT dataflows contrasted with taoFPGA's fixed fused pipeline. 13

List of Tables

Table I. transformer_block_axi_top AXI4-Lite Register Map. 10
Table II. Post-Route Signoff Metrics, Before/After the DSP-Inference Fix. 12
Table III. Kernel Benchmark Results, PYNQ-Z2 vs. ARM Cortex-A9. 13
Table IV. Comparison with Literature FPGA Transformer Accelerators. 14

PRELIMINARIES

This section briefly introduces the three concepts the rest of the paper builds on, for a reader not already familiar with them.

A. Transformer Self-Attention

A Transformer block processes a sequence of N token embeddings by letting every token attend to every other token [1]. Each token's embedding is linearly projected into a query (Q), key (K), and value (V) vector; attention weights are the scaled dot product of each query against every key, normalized with Softmax into a probability distribution over the sequence, and the output for each token is the value vectors weighted by that distribution. Because every token attends to every other token, this costs $O(N^2)$ multiply-accumulate operations in the sequence length — the quadratic self-attention cost referenced throughout this paper. A position-wise feed-forward network — two dense layers with a GELU nonlinearity — is then applied independently to each token. Vision Transformers (ViT) apply this identical mechanism to a sequence of flattened image patches rather than word tokens [7]. taoFPGA implements the two computationally dominant primitives in this pipeline — the dense matrix multiplications and their associated Softmax/GELU nonlinearities — as dedicated fixed-point hardware, fused into one streaming pipeline (Section II).

B. Fixed-Point (INT8) Quantization

Running a Transformer in the 32-bit floating-point format it is normally trained in is prohibitively expensive on an embedded FPGA: every multiply-accumulate costs more logic, and every stored value costs 4× the memory bandwidth of an 8-bit alternative. Quantization instead represents each value as a signed 8-bit integer together with a shared, per-tensor scale factor (a power of two in this design), so a real value x is approximated as round(x/scale). Two INT8 values multiply to a result with roughly double the bit width of the inputs; to keep every tensor in the pipeline at a consistent INT8 width, the accumulated product must be rescaled back down by an appropriately chosen shift and re-clipped to the representable [−128, 127] range — the round-half-up, saturate-to-INT8 operation formalized as Eq. 1 and implemented in right_shifter.v. The central engineering trade-off is choosing each stage's scale/shift so the rescaled result neither saturates (losing dynamic range) nor rounds away the signal (losing precision); Section II.C and Appendix A.3 describe two concrete bugs this project encountered getting that calibration right.

C. Systolic Arrays for Matrix Multiplication

A systolic array computes matrix multiplication with many small processing elements (PEs) — each capable of one multiply-accumulate per cycle — arranged in a regular 2D grid, streaming operands through the grid in a staggered rhythm so each PE only ever exchanges data with its immediate neighbors, never a global memory or a distant PE. This locality is what makes the design scale: adding more PEs adds compute without adding long, power-hungry wires. taoFPGA uses a weight-stationary variant: each PE loads and holds one weight value for the duration of a matrix multiplication while feature values stream through the array row by row, maximizing reuse of each loaded weight and matching the Transformer's own workload pattern, where the same projection weight matrix is reused across every token in the input sequence. Section II.A and Appendix A.7 describe the specific 16×16 instantiation of this idea and the physical-implementation work needed to map it efficiently onto the FPGA's dedicated DSP48E1 hard multiplier blocks.

I. INTRODUCTION & MOTIVATION

A. The Edge AI Compute Dilemma

Transformer models now dominate both natural-language processing and computer vision, but their computational profile — quadratic self-attention and large dense projections in every encoder layer — sits uneasily on embedded edge platforms, where DRAM bandwidth, on-chip memory, and power budget are all scarce simultaneously [1]. Prior FPGA accelerators for transformer-family models converge on a small set of recurring design choices — systolic or array-of-PE matrix multiplication, fixed-point quantization, and dedicated non-linear activation units — to close this gap [2]–[5]. This project targets that same design space on a resource-constrained, commodity Zynq-7020 SoC.

B. Design Intent & Methodology

taoFPGA was developed bottom-up: each arithmetic core — the systolic matrix multiplier, the Softmax normalization engine, and the GELU activation unit — was designed and verified in isolation against a real-valued golden reference

before being composed into a single streaming pipeline (MatMul → Softmax → GELU), integrated into a complete SoC, physically implemented, and finally evaluated against real Vision Transformer (ViT) workload shapes on physical hardware. A recurring methodological principle, arrived at empirically over the course of the project, governs every phase: no layer of the system — golden model, integration script, board, register map, or DMA transfer — may be assumed correct on the strength of another layer's correctness. Each was checked directly against its own ground truth.

C. Key Contributions & Paper Outline

(1) A 16×16 INT8 weight-stationary systolic array fused with dedicated fixed-point Softmax and GELU engines into a single streaming pipeline.

(2) A tolerance-bounded verification methodology carried consistently from SystemVerilog RTL simulation through a Python/NumPy golden model validated against physical silicon.

(3) Identification and correction of a synthesis-level DSP-inference defect, reducing LUT utilization by 49.3% and total on-chip power by 3.1% at signoff.

(4) First physical hardware validation of the design on a PYNQ-Z2, matching simulation to zero error across 32,000 output elements at full scale.

(5) A kernel-level hardware/software benchmark against a real, numerically validated ViT-Tiny baseline, reporting 70.02×-97.73× measured speedup and a total energy efficiency of 2.25 GOPS/W at peak throughput, together with an explicit architectural analysis of why the accelerator's fixed pipeline does not map onto any single ViT operation.

(6) A quantified account of the design's open hardware constraints — principally a 64 KB DMA transfer ceiling — and the re-synthesis path identified to resolve them.

The remainder of this paper is organized as follows. Section II details the core hardware architecture and its fixed-point formulation. Section III covers RTL verification. Section IV describes SoC integration on the PYNQ-Z2. Section V reports physical implementation and signoff. Section VI presents experimental evaluation and hardware bring-up. Section VII discusses remaining bottlenecks and future work, and Section VIII concludes.

II. CORE HARDWARE ARCHITECTURE & MATHEMATICAL FORMULATION

A. Systolic Matrix Multiplication Core

The matrix multiplication core (MM_ultra) is a weight-stationary 16×16 systolic array of INT8 processing elements (256 MACs total), fed by dedicated feature and weight input buffers (MM_in_buffer, MM_buffer) that stream operands into the array in A_size-wide parallel beats. Runtime-configurable block-count registers (F_width_block_num, W_width_block_num) parameterize the contraction and output dimensions, both constrained to multiples of the array width. Accumulated products are rounded and saturated to INT8 by a dedicated shifter: the design rounds half-up on an arithmetic right shift — equivalently, add 2^(shift−1) then shift, or inspect the most-significant discarded bit — before clamping to [−128, 127]. Both formulations were confirmed mathematically identical during hardware bring-up (Section VI) when the software golden model was cross-checked line-by-line against this shifter's RTL. Formally, for a pre-shift accumulator value z and a runtime-configurable shift ∈ $\mathbb{Z}^+$, the quantization operator realized in hardware is

$$\mathrm{Quant}(z) = \mathrm{clip}\left(\left\lfloor (z + 2^{\,\mathrm{shift}-1}) \cdot 2^{-\mathrm{shift}} \right\rfloor,\ -128,\ 127\right) \quad (1)$$

where ⌊·⌋ denotes floor (equivalently, arithmetic right shift for two's-complement operands) and clip(·, lo, hi) saturates its argument to [lo, hi]. The degenerate case shift = 0 is handled as a direct clip of z with no rounding term. Listing 1 excerpts the actual RTL realization of (1) in right_shifter.v, which computes the round term by inspecting the discarded bit directly rather than via an explicit add-then-shift, the two formulations having been confirmed identical as noted above.

```
// right_shifter.v -- round-half-up + saturate to
INT8
assign temp1_out = data_in >>> shift;
assign temp2_out =
    data_in[shift-1] ? temp1_out + 1 : temp1_out;
// under_min/over_max: range checks on temp2_out
case ({under_min, over_max})
    2'b10: data_out = {1'b1, {(W-1){1'b0}}}; // -128
    2'b01: data_out = {1'b0, {(W-1){1'b1}}}; // +127
    default: data_out = temp2_out[W-1:0];
endcase
```

Listing 1. Round-half-up and saturate operator, right_shifter.v (Eq. 1).

Fig. 1 details one PE's internal datapath (PE.v): the stationary weight register reg_w is loaded once via set_w and held for the duration of a matrix multiplication, while x_in streams through a registered pass-through (x_out) to the next PE to the east and simultaneously feeds the multiplier; the multiplier's product accumulates with psum_in from the PE to the north into a 20-bit signed register (2·data_width + $\log_2$(array_m) for this 16-row array) that is the exact register Appendix A.7's DSP-inference fix targets, and is forwarded south as psum_out.

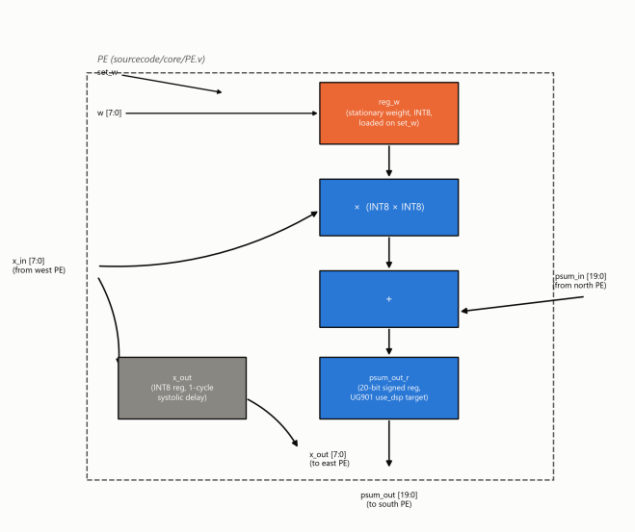

Fig. 1. One PE's internal datapath: stationary weight register, INT8 multiply, 20-bit accumulate, and systolic x/psum propagation.

B. Fixed-Point Non-Linear Arithmetic Engines

Softmax normalization (Softmax_control / Softmax) is a three-pass scalar architecture operating on one buffered row at a time: pass one finds the row maximum via a running comparator; pass two streams the row a second time, computing and accumulating exp(x−max) via a base-2 exponential approximation (a rational scaling of x by $\log_2(e)$, split into integer and fractional components with linear interpolation of the fractional power-of-two term); pass three streams the row a third time, computing the final normalized value exp(x−max−ln(Σ)) using a second instance of the same exponential unit combined with a piecewise logarithm approximation (Ln_module) to avoid an explicit division. Formally, for a row x with elements x_i, the three passes compute

$$m = \max_i(x_i)$$
$$S = \sum_i 2^{(x_i - m)\log_2 e}$$
$$y_i = 2^{((x_i - m) - \ln S)\log_2 e} \quad (2)$$

an exact base-2 reformulation of $\text{softmax}(x)_i = \exp(x_i - m)/\Sigma$ that matches the hardware's own $\log_2(e)$-scaled exponential and Ln_module log-domain division-avoidance strategy exactly, rather than the textbook exp/ln formulation alone. Fig. 2 depicts the resulting finite-state control flow: each of the three passes streams the full N-element row exactly once, strictly sequentially, for a total latency of 3N cycles per row — the structural origin of the serial Softmax bottleneck quantified in Section VII.B.

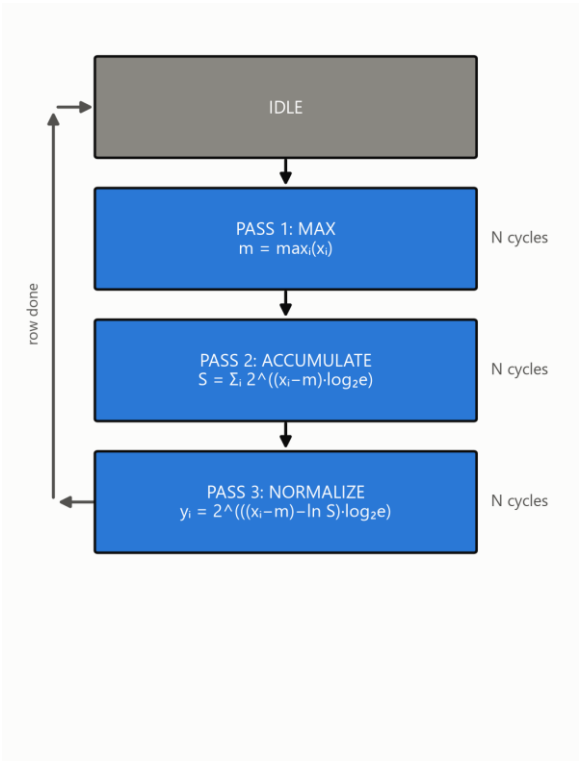

Fig. 2. Softmax_control's 3-pass FSM and its per-pass equations (Eq. 2).

The GELU activation (gelu.v) is a two-segment piecewise-linear approximation: inputs beyond a fixed saturation threshold

pass through unmodified or clamp to zero according to sign, and inputs within that range are computed via a linear correction term generated by a dedicated interpolation submodule (lin.v).

C. Numerical Quantization Strategy

Features, weights, and pipeline output are all INT8. The matmul accumulates in extended precision before the shift-and-saturate step described above; Softmax and GELU each carry independent runtime-configurable fixed-point scale registers (softmax_scale_in, softmax_scale_out, gelu_scale), with softmax_scale_out and gelu_scale constrained equal so that the two activation stages interpret the shared int8 boundary between them consistently.

III. RTL VERIFICATION & BEHAVIORAL SIMULATION (CADENCE XCELIUM)

A. Testbench Architecture & Golden Reference Model

Four SystemVerilog testbenches (gelu_tb, Softmax_top_tb, MM_Ultra_tb, and an integration-level transformer_block_tb) each implement a real-valued ($real) golden reference directly in SystemVerilog: an integer matmul with the same round-half-up-and-saturate rule as the RTL shifter, a numerically stable softmax via $exp, and a tanh-approximation GELU via $tanh. The integration testbench chains all three references to validate the full streaming pipeline against one coherent golden model.

B. Simulation Results & Numerical Precision Verification

Verification is tolerance-bounded throughout, not exact-match: unit-level tests compare against a scale-dependent LSB threshold, and the full-pipeline integration test uses a 6-LSB tolerance at the shared Softmax-output/GELU scale to absorb error compounding across the two INT8 quantization boundaries. At full scale (200×96×160), the integration run recorded zero of 32,000 output elements outside tolerance, with an 18,083-cycle end-to-end pipeline latency.

Fig. 3 illustrates the AXI4-Stream handshake and the 3-pass timing structure a feature row actually traverses on ingest — shown as a protocol-structure diagram rather than a captured simulation trace, since visualizing a specific signal-level EDA waveform faithfully would require an actual Xcelium/SimVision trace export, not a reconstruction.

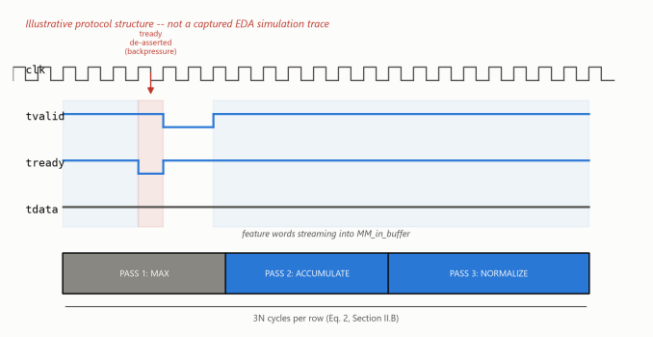

Fig. 3. Illustrative AXI4-Stream handshake and 3-pass Softmax timing structure (protocol shape, not a captured EDA trace).

C. Pre-Silicon Logic Debugging

Three representative issues surfaced during simulation bring-up: a stimulus-side forward-reference bug in a testbench control signal (Appendix A.2); an Xcelium elaboration-time memory blowup traced to a generate-loop-unrolled per-scalar wire array that scaled combinatorially with matrix size, resolved by replacing it with an on-demand packing function (Appendix A.1); and a datapath port declared as output reg rather than output wire, which was silently dropped during elaboration rather than flagged as an error. A fourth, logically distinct issue — a documented but never-closed port-width mismatch between Softmax's and GELU's scale registers — is detailed separately in Appendix A.3. Each entry gives the full investigation and fix, not just the one-line summary above.

IV. SOC INTEGRATION & HARDWARE PLATFORM DESIGN (PYNQ-Z2 / ZYNQ-7020)

A. SoC Architecture

The accelerator is wrapped as an AXI4-Lite-controlled, dual AXI4-Stream-slave / single AXI4-Stream-master peripheral (transformer_block_axi_top) and integrated alongside the Zynq-7020's dual-core ARM Cortex-A9 processing system, an AXI SmartConnect interconnect, and three independent AXI Direct Memory Access engines (feature, weight, and result channels) via Xilinx IP Integrator. Fig. 4 shows the complete top-level datapath: the PS7 configures the pipeline over AXI-Lite and streams feature and weight tensors in via two MM2S DMA channels, while a third, independent S2MM channel returns the GELU stage's output; internally, width-adapter glue logic

bridges each stage's differing datapath width, exactly as detailed in Section IV.B.

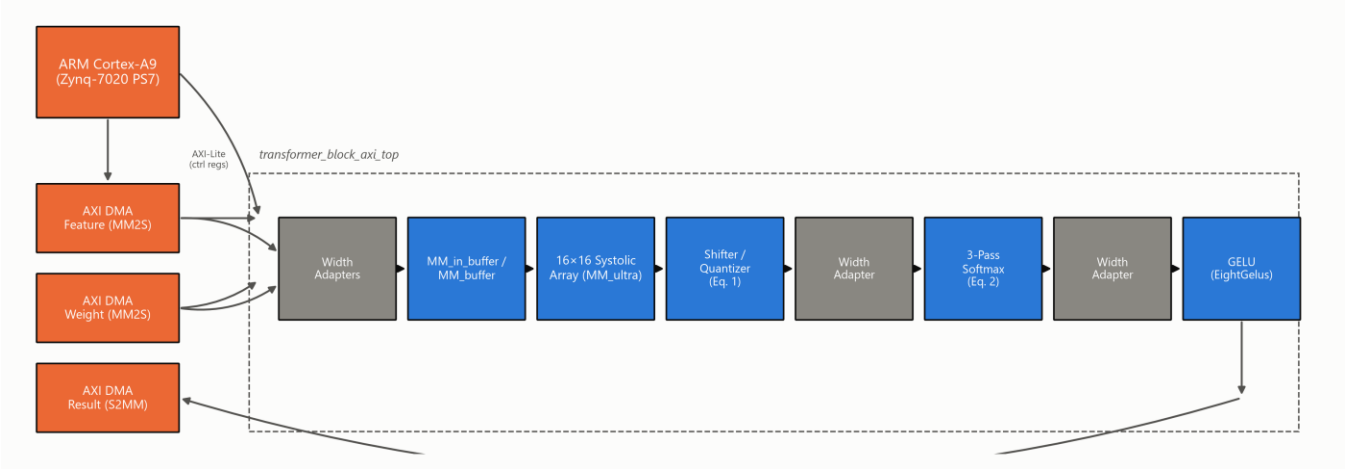

Fig. 4. Complete top-level SoC architecture: PS7, three AXI DMA channels, and the transformer_block_axi_top internal pipeline.

Table I lists transformer_block_axi_top's complete AXI4-Lite register interface, sourced directly from transformer_block_axi.v's own header comment: eight 32-bit words at addr[4:2], seven read/write configuration registers plus one read-only status bit.

TABLE I. transformer_block_axi_top AXI4-LITE REGISTER MAP

Addr	Register	Bits	R/W	Description
0x00	mm_shift_in	10	RW	Matmul output right-shift amount (Eq. 1)
0x04	mm_F_length_in	10	RW	Feature matrix row count (F_length)
0x08	mm_F_width_block_num_in	5	RW	Feature matrix width, in A_size-wide blocks
0x0C	mm_W_width_block_num_in	5	RW	Weight matrix width, in A_size-wide blocks
0x10	softmax_scale_in	4, signed	RW	Softmax input fixed-point scale
0x14	softmax_scale_out	4	RW	Softmax output scale (= gelu_scale, App. A.3)
0x18	gelu_scale	4	RW	GELU input/output fixed-point scale
0x1C	status	1	RO	bit0 = softmax_to_gelu_fifo_overflow (live, not latched)

B. Physical Adaptations for Board Deployment

Integration surfaced two categories of adaptation beyond the standalone core. First, a synthesis-time design rule check flagged over 40 instances of block-RAM control logic driven by asynchronously reset registers inside the input/output buffers — a pattern Xilinx's own methodology documentation warns can corrupt memory contents outside static timing analysis' visibility. This was resolved across twelve RTL files by converting every affected sensitivity list from asynchronous to synchronous reset (Appendix A.5 gives the full three-round root-cause chase). Second, bridging the array's A_size-wide parallel datapath to Softmax's serial, one-scalar-per-cycle

interface (and back to GELU's narrower parallel lanes) required bespoke AXI4-Stream width converters and a purpose-built row-boundary signal, since the matmul core's own frame-completion signal marks only the end of the entire matrix, not each row — the same integration phase that first revealed the full pipeline had never actually been AXI-wrapped (Appendix A.6), following the parameter-audit discipline established against the adopted integration script (Appendix A.4).

C. Software Infrastructure for Hardware Bring-Up

To extend the tolerance-bounded methodology of Section III to physical silicon, the SystemVerilog golden references were ported to a bit-faithful Python/NumPy implementation, cross-checked against the RTL shifter's exact rounding behavior rather than assumed equivalent, and used to validate hardware output read back over the board's PYNQ Python driver.

V. PHYSICAL IMPLEMENTATION, P&R, AND SIGN-OFF OPTIMIZATIONS

A. Baseline Physical Sign-Off

The first complete post-route signoff of the full SoC closed timing at the 100 MHz target with a Worst Negative Slack (WNS) of +0.361 ns and a Vivado-estimated total on-chip power of 1.741 W (1.590 W dynamic, 0.151 W static).

B. The DSP Inference Gap & Attribute Resolution

Post-route utilization at this baseline was LUT-bound rather than DSP-bound: only 13 of 220 available DSP48E1 slices (5.91%) were in use while Slice LUTs sat at 56.91%, despite the 256-MAC systolic array being the design's dominant arithmetic structure — a strong signal that the array's multiply-accumulate operations were being synthesized into general LUT/CARRY4 fabric instead of dedicated DSP hard macros. Root-causing traced this to a synthesis attribute scoped at the wrong granularity relative to Vivado's register-level DSP inference rules (cf. Xilinx UG901 [6]): as Listing 2 shows, attaching use_dsp to the enclosing always block is a silent no-op — synthesis produced byte-identical LUT/DSP/CARRY4 counts regardless of its value — whereas attaching it to the register declaration that actually holds the multiply-accumulate result is what Vivado's inference pass recognizes. Appendix A.7 gives the full investigation, including why this specific attribute-attachment rule is easy to miss from the UG901 text alone.

```
// PE.v -- BEFORE: attribute on the always block
(no-op)
(* use_dsp = "yes" *)
always @(posedge clk) begin
    psum_out <= psum_in + x_in*reg_w;
end

// AFTER: attribute on the register declaration (per
UG901)
(* use_dsp = "yes" *) reg signed
    [2*data_width+log2_array_m-1:0] psum_out_r;
always @(posedge clk)
    psum_out_r <= psum_in + x_in*reg_w;
```

Listing 2. The UG901 DSP-inference attribute-placement fix, PE.v.

Correcting the attribute placement alone would map all 256 PEs to DSP48E1 hard macros — more than the xc7z020's 220-slice budget once Softmax and GELU's own arithmetic (≈13 DSPs) is accounted for. PE_array.v therefore parameterizes the split per PE row (NUM_DSP_ROWS = 12 of 16), mapping exactly 192 of 256 PEs to DSP48E1 and leaving the remaining 4 rows (64 PEs) LUT/CARRY4-mapped — a deliberate, budget-sized partial mapping, not an all-or-nothing switch. This reduced full-SoC post-route Slice LUTs by 49.3% (30,276 → 15,363) and CARRY4 primitives by 61.8% (4,874 → 1,862), brought DSP48E1 usage to 205 of 220 (192 from the array plus Softmax/GELU's ≈13), and reduced total on-chip power by 3.1% to 1.687 W.

C. Physical Congestion vs. Timing Trade-Off

The same fix concentrated 205 of 220 DSP48E1 slices (93.18% of the device's entire DSP column capacity, 15 slices of headroom remaining) into a much denser physical footprint than the previous LUT-diffuse implementation. Post-route WNS correspondingly tightened from +0.361 ns to +0.293 ns (≈19% less margin, timing still met) — an expected, explicitly-anticipated trade-off between area/power efficiency and routing congestion around a densely packed DSP column, not a regression.

[Figure Placeholder: Vivado Device View / Floorplan — Illustrating physical DSP48E1 column clustering (205/220 slices, 93.18% utilization) and dense local interconnect routing explaining the WNS trade-off (+0.293 ns). Not included: this session has no saved screenshot from the actual signed-off Vivado run to insert here

honestly; a real capture (Device window, DSP48E1 primitive filter highlighted) should replace this box before final submission.]

D. Bitstream & Hardware Platform Sign-Off

The signed-off, DSP-corrected checkpoint passed pre-bitstream design rule checking with zero errors and generated a bitstream with zero warnings and zero critical warnings, exported as design.bit and a fixed hardware platform archive (design.xsa) for downstream software and PYNQ deployment.

TABLE II. POST-ROUTE SIGNOFF METRICS, BEFORE/AFTER THE DSP-INFERENCE FIX

Metric	Before fix	After fix	Δ
Slice LUTs (post-route, full SoC)	30,276 / 53,200 (56.91%)	15,363 / 53,200 (28.88%)	−49.3%
CARRY4 (post-route, full SoC)	4,874 / 13,300	1,862 / 13,300	−61.8%
DSP48E1	13 / 220 (5.91%)	205 / 220 (93.18%)	+192
WNS @ 100 MHz	+0.361 ns	+0.293 ns	−19% margin
Total on-chip power	1.741 W	1.687 W	−3.1%

VI. EXPERIMENTAL EVALUATION & HARDWARE BRING-UP

A. Benchmarking Methodology

Hardware kernel throughput was benchmarked on the physical PYNQ-Z2 against two matrix shapes representative of ViT-Tiny's real layer dimensions (embedding dimension 192, sequence length 197): a 192×192 projection-sized kernel and a 192×320 MLP-shaped kernel, each compared against an equivalent computation of the identical fused matmul-Softmax-GELU operation executed on the board's ARM Cortex-A9.

B. Performance Metrics & Acceleration

The hardware kernel achieved 2.487 GOP/s and 3.795 GOP/s on the two benchmarked shapes respectively, against 0.036 and 0.039 GOP/s on the CPU — measured speedups of 70.02× and 97.73× for the identical operation. At the 3.795 GOP/s peak (192×320, MLP-shaped kernel), against the post-route signoff power figures of Table II (1.687 W total on-chip, 1.538 W dynamic), the design achieves a total energy efficiency of 2.25 GOPS/W and a dynamic energy efficiency of 2.47 GOPS/W. These efficiency figures combine a live-measured kernel throughput with Vivado's post-route, activity-based power estimate for the full SoC design — not a physically instrumented power reading synchronized to the benchmark run itself — and should be read as a signoff-grade estimate accordingly. Figs. 5–7 present latency, throughput, and speedup for both shapes.

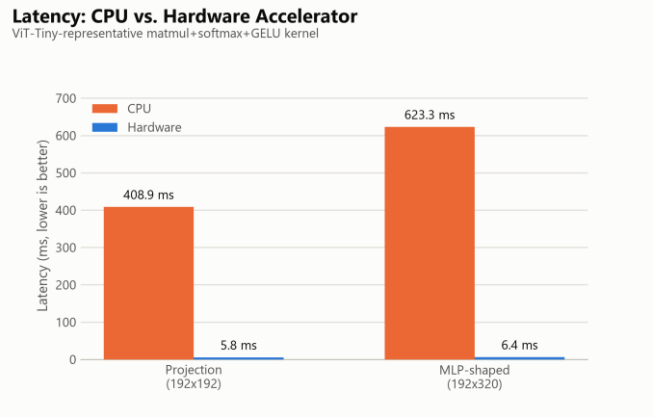

Fig. 5. Latency, CPU vs. hardware, both benchmarked shapes.

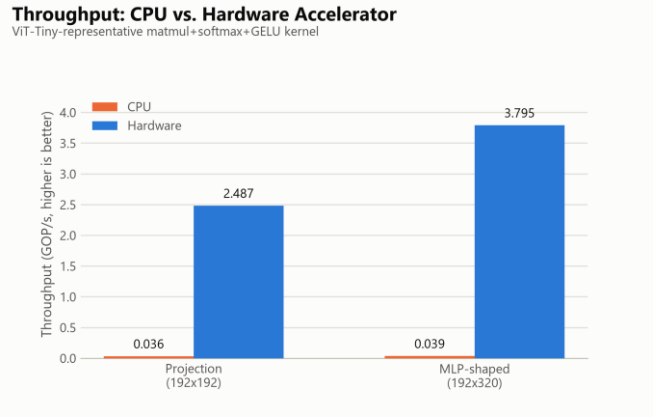

Fig. 6. Throughput (GOP/s), CPU vs. hardware, both shapes.

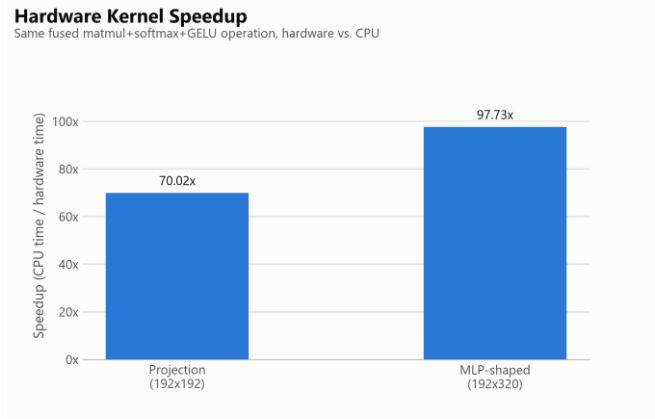

Fig. 7. Measured hardware kernel speedup over CPU.

TABLE III. KERNEL BENCHMARK RESULTS, PYNQ-Z2 vs. ARM CORTEX-A9

Shape	HW latency	CPU latency	Speedup
192×192 (projection)	5.84 ms	408.9 ms	70.02×
192×320 (MLP-shaped)	6.38 ms	623.3 ms	97.73×

C. Physical Silicon Verification

Independent of the kernel benchmark above, the full pipeline was validated at the exact scale exercised in Section III's simulation (200×96×160, an 18,083-cycle simulated end-to-end latency): the physical hardware run reproduced the software golden model with zero of 32,000 output elements outside the established tolerance bound, confirming that behavioral simulation and physical silicon agree at the design's originally verified scale.

D. Workload Characterization vs. ViT Networks

A software-only ViT-Tiny (patch16, 224×224, 5.7M parameters [7], [8]) forward pass, reimplemented from scratch in NumPy and validated against its reference PyTorch implementation to a maximum logit difference of 1.02×10^{-5}, reproduced an identical top-1 prediction with 87.26% confidence on a sample image — a single-sample confidence score, not a classification accuracy measured over a labeled evaluation set. Critically, the accelerator's fixed matmul→Softmax→GELU pipeline does not structurally correspond to any single ViT-Tiny or ViT-Base operation: multi-head self-attention consists of matrix multiplications and one Softmax with no GELU stage, while the MLP block consists of a matrix multiplication and GELU with no Softmax between them. Substituting the accelerator into either would silently apply an activation or normalization step the real operation does not include, corrupting the result. This architectural mismatch — verified directly against the RTL rather than assumed — is the reason Section VI.A–B report a kernel-level throughput benchmark rather than an end-to-end accelerated ViT inference claim.

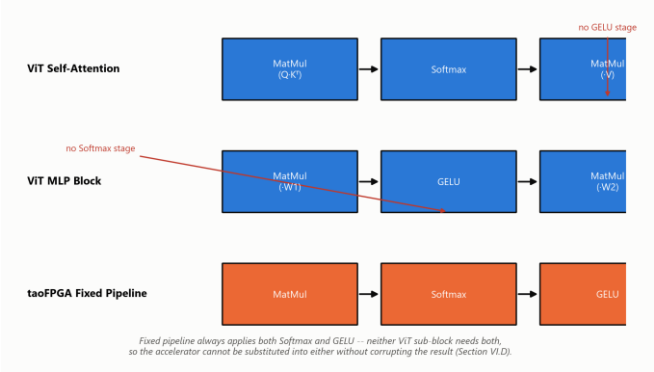

Fig. 8. Real ViT self-attention and MLP dataflows contrasted with taoFPGA's fixed fused pipeline.

E. Hardware Bring-Up Observations & Edge Cases

Two open hardware findings emerged during bring-up. First, a root-caused defect in the input buffer's row-replay address counter (MM_in_buffer): for a single-row configuration (F_length = 1), the counter advances to 1 immediately on start but its wrap-to-zero condition checks against F_length−1 = 0, a value it can then never reach again, causing the buffer to read stale data indefinitely (Appendix A.11 traces the full diagnostic path to this root cause). Second, a related but distinct DMA hang observed at a two-row configuration was traced far enough to rule out the same cause but not far enough to identify its root (Appendix A.10 covers the methodical, cheapest-checks-first process that narrowed the search before this residual gap was reached); both are confined to matrix shapes far smaller than any configuration this design was validated at (Section III.B, VI.C), and the codebase was frozen with both documented as open items (Section VII) rather than resolved in this scope.

F. Comparative Analysis with Prior FPGA Transformer Accelerators

Table IV situates taoFPGA relative to two literature FPGA transformer accelerators, ME-ViT [3] and FTRANS [4], restricted to figures independently confirmed from each paper's own reported text rather than secondary summaries. All three designs converge on the same architectural pattern noted in Section I.A — fixed-point quantization paired with a DSP-array-dominated datapath, each running its target device's DSP budget close to saturation.

TABLE IV. COMPARISON WITH LITERATURE FPGA TRANSFORMER ACCELERATORS

Design	Target FPGA (total DSPs)	Precision	DSP utilization	Reported efficiency
taoFPGA (this work)	Zynq-7020 (220)	INT8	205 / 220 (93.2%)	2.25 GOPS/W total; 70.0×–97.7× vs. on-chip ARM Cortex-A9
ME-ViT [3]	Alveo U200 (5,867)	not stated in source	1,024 / 5,867 (17.5%)	4.00× power efficiency vs. GPU; 9.22×–17.89× memory-bandwidth gain
FTRANS [4]	VCU118 (6,840)	16-bit fixed-point + BCM compression	5,647–6,531 / 6,840 (82.6–95.5%)	81×/8.80×/2.44× energy efficiency vs. CPU/GPU/Jetson TX2

A direct throughput or GOPS/W ranking across all three would misrepresent the comparison: ME-ViT and FTRANS target datacenter-class FPGAs (5,867 and 6,840 DSPs respectively) benchmarked against GPU/CPU baselines, whereas taoFPGA targets a resource-constrained embedded SoC with 220 total DSPs — a difference of more than 25× in available DSP budget alone — benchmarked against its own on-chip ARM Cortex-A9. This table is offered to situate taoFPGA's design choices within the same architectural family as prior work, not to claim a device-independent performance ranking; the same discipline against overclaiming a comparison already governs the ViT workload-mismatch analysis of Section VI.D.

VII. ENGINEERING DISCUSSION, BOTTLENECKS & FUTURE WORK

A. DMA Buffer Boundary Constraints

The design's three AXI DMA engines were synthesized with a 16-bit transfer-length register, capping any single DMA transfer at 65,536 bytes. This directly prevented benchmarking the real 768-wide ViT-Tiny MLP hidden dimension (a 192×768 weight buffer alone requires 147,456 bytes) and, more broadly, blocks batch processing entirely: streaming even two images' feature matrices against a shared weight matrix already overflows the ceiling for every benchmarked shape in this work. Splitting a transfer across multiple smaller DMA calls is not a safe software workaround with the current RTL, since the input buffer's write-address counters reset on the stream's tlast signal, which the design's direct-register DMA mode asserts at the end of every transfer issued — not only a genuinely final one — corrupting rather than merely truncating a chunked transfer. Appendix A.13 gives the full account of why this specific workaround was rejected, including the RTL trace that ruled it unsafe.

B. Serial Softmax Throughput Bottleneck

The systolic array produces A_size parallel INT8 results per cycle, while the Softmax engine consumes and produces exactly one scalar per cycle across three full passes over each row. This structural mismatch — a fully parallel two-dimensional compute stage feeding a one-dimensional serial normalization stage — is a standing throughput bottleneck independent of clock frequency or DSP utilization, and is the most direct architectural target for a future parallel or pipelined Softmax redesign.

C. Architectural Roadmap

Two concrete next steps follow directly from Sections VII.A–B: (i) replacing direct-register DMA with genuine Scatter-Gather DMA (or, more simply, re-synthesizing with a wider transfer-length register), giving explicit per-descriptor frame-boundary control and removing the single-transfer ceiling that currently blocks both full-width MLP evaluation and batch processing; and (ii) decoupling the matmul, Softmax, and GELU

stages into independent streaming nodes with real backpressure support, addressing both the serial Softmax bottleneck and a separately documented limitation in the GELU stage's lack of an internal skid buffer.

VIII. CONCLUSION

taoFPGA demonstrates a complete hardware/software co-design lifecycle for a quantized transformer processing core, from fixed-point mathematical formulation and tolerance-bounded RTL verification through physical signoff, real hardware bring-up, and a kernel-level benchmark against a genuinely validated software ViT baseline. Beyond the measured results — a 49.3% LUT reduction and 3.1% power reduction from a single synthesis-level fix, zero-error silicon validation at full verified scale, and up to 97.73× kernel speedup over an embedded ARM CPU — the project's governing discipline throughout was to verify every layer of the system against its own ground truth rather than inherit correctness from an adjacent layer, and to report every limitation — architectural, numerical, or a hardware transfer ceiling — as precisely as the result being claimed. The 64 KB DMA ceiling and its Scatter-Gather resolution path, together with the serial Softmax bottleneck, define a clear and quantified roadmap for the next iteration of this work.

REFERENCES

[1] A. Vaswani et al., “Attention Is All You Need,” in Proc. NeurIPS, 2017.

[2] B. J. Kang, H. I. Lee, S. K. Yoon, Y. C. Kim, S. B. Jeong, S. J. O, and H. Kim, “A survey of FPGA and ASIC designs for transformer inference acceleration and optimization,” J. Systems Architecture, vol. 155, art. 103247, Oct. 2024.

[3] K. Marino, P. Zhang, and V. K. Prasanna, “ME-ViT: A Single-Load Memory-Efficient FPGA Accelerator for Vision Transformers,” in Proc. IEEE 30th Int. Conf. High Performance Computing, Data, and Analytics (HiPC), 2023.

[4] B. Li, S. Pandey, H. Fang, Y. Lyv, J. Li, J. Chen, M. Xie, L. Wan, H. Liu, and C. Ding, “FTRANS: Energy-efficient acceleration of transformers using FPGA,” in Proc. ACM/IEEE Int. Symp. Low Power Electronics and Design (ISLPED), 2020.

[5] Y. Chen, T. Li, X. Chen, Z. Cai, and T. Su, “High-Frequency Systolic Array-Based Transformer Accelerator on Field Programmable Gate Arrays,” Electronics, vol. 12, no. 4, art. 822, 2023.

[6] AMD/Xilinx, “Vivado Design Suite User Guide: Synthesis (UG901),” AMD, San Jose, CA, USA.

[7] A. Dosovitskiy et al., “An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale,” in Proc. ICLR, 2021.

[8] R. Wightman, “PyTorch Image Models (timm),” GitHub repository, 2019. [Online]. Available: https://github.com/huggingface/pytorch-image-models

Appendix A: Engineering Problems Encountered and Their Solutions

The verified results in the body of this report are the outcome of a long, iterative debugging process, not a linear implementation. This appendix records the real, individually significant problems found and solved along the way, condensed from the project's full chronological engineering log (report/project_story.md), specifically so this report is sufficient on its own to answer detailed, in-depth questions about how the project's key results were actually reached.

A.1 Elaboration-Time Memory Blowup From `generate`-Unrolled Test Arrays

Running the matmul testbench at full scale (200×96×160) crashed the entire simulation session -- not just the job, the whole host -- during Xcelium's native-code-generation phase.

Investigation. The partial log pointed at native-code generation for the testbench module itself. Hypothesis: `x_in_array`/`y_in_array`/`z_hard_array` were built with a SystemVerilog `generate` loop that elaborates one hardware object per scalar array element -- fine for genuinely parallel structures, catastrophic for a large lookup table. Rather than guess and risk a second crash, a resource-monitored, kill-switch-protected staged rollout ($16^3 \rightarrow 32^3 \rightarrow 64^3 \rightarrow 128^3$, polling combined process RSS every 2 seconds with a hard kill at 5,500MB) confirmed memory grew worse than quadratically -- 59MB → 76MB → 482MB → >5,900MB and still climbing at 128^3.

Fix. Replaced the pre-elaborated arrays with `automatic` SystemVerilog functions (`pack_x_word`, `pack_y_word`, `unpack_z_word`) that compute each word at runtime, on demand, instead of materializing every possible word as a separate hardware object. Full scale now runs in ~51MB / ~4 seconds, with functional output confirmed bit-for-bit unchanged (identical 2,894-cycle latency figure before and after).

Why this matters. *This is a real distinction between synthesizable-style hardware description and software-style data structures written in the same language: a `generate` loop is an elaboration-time construct, and using it to hold data (as opposed to genuinely parallel structure) doesn't scale. It's also a strong example of diagnosing a scaling problem with a bounded, safety-netted experiment instead of guessing -- the same protocol was reused for every large simulation/synthesis run for the rest of the project.*

project_story.md §6-9

A.2 A Silent Compile Bug Hidden by an Untested Code Path

The very first attempt to actually run the matmul testbench (after weeks lost to an unrelated infrastructure problem) hit an immediate compile error: an undeclared identifier, `start_trans`.

Investigation. `git blame` showed a later commit had added instrumentation code that referenced `start_trans` above the line that actually declared it -- a pure ordering bug. The testbench had been silently uncompilable since that commit, but nothing had ever gotten far enough to hit it, because every previous attempt to run it had failed earlier, for an unrelated infrastructure reason.

Fix. Moved the declaration above its first use. No logic change.

Why this matters. *A working-looking pipeline can hide a real, waiting compile bug indefinitely if nothing ever exercises the path that hits it. The practical lesson: don't assume a code path is fine just because nothing has complained about it yet -- absence of failure isn't evidence of correctness if the path was never actually reached.*

project_story.md §6

A.3 A Documented, Deliberately-Deferred Bug That Was Still a Real Bug

`Softmax_control`'s scale output port was 4 bits wide (valid range 7-12) but `gelu.v`'s corresponding scale input port was only 3 bits wide (max value 7) -- the two IPs could only agree at exactly scale=7. This was already known and explicitly flagged in a code comment, left unfixed by whoever wrote the original integration blueprint.

Investigation. Reading `gelu.v`'s actual shift logic showed this wasn't a trivial port-width bump: the code assumed the input scale maxed out at 7 and had no branch for anything higher, so widening the port alone would have silently mis-shifted any newly representable value 8-15 as if it were 7 -- a correctness bug, not just a missing feature.

Fix. Widened the scale port through all three affected modules to 4 bits, added the missing shift-direction branch in `gelu.v`, and extended the GELU testbench's randomized range from 0-6 to 0-11 (the old range never even reached the boundary value 7 that used to be the only legal one).

Why this matters. *A code comment that says "known limitation, not fixed here" is not the same as "already handled" -- it's worth actively hunting these down rather than trusting they'll be addressed later by someone else. It's also a good interview example of reading an interface's actual implementation, not just its declared width, before trusting a narrow-looking fix.*

project_story.md §10, §12

A.4 Six Silent Parameter Mismatches Between Verified RTL and an Adopted Integration Script

Reviewing the externally-adopted Vivado integration script (`prj.tcl`, originally exported from an earlier Vivado session, never maintained against this project's own verified RTL) ahead of synthesis, one visible mismatch was found: the systolic array size was configured as 24, but every verified testbench uses 16.

Investigation. Rather than fix the one visible value and move on, every one of the wrapper module's ten RTL-facing parameters was cross-referenced against the exact value the verified testbenches use. The audit found six mismatches, not one -- and the two most dangerous ones (a shift-width and a block-count-width parameter) were never even mentioned in the integration script at all, silently inheriting a stale wrapper default with no trace of the problem visible anywhere in the file someone would normally review.

Fix. Corrected all six parameters, plus two downstream AXI-Stream width-converter settings that had been sized to match the wrong array size.

Why this matters. *The core principle established here (and used for the rest of the project): verified RTL and testbenches are the source of truth, and any adopted external tooling gets adapted to match them -- never assumed correct just because it was machine-generated by the official tool. The specific lesson worth repeating in an interview: the values that are silently missing from a config (falling back to some default) are often more dangerous than the values that are visibly wrong, because nothing prompts a reviewer to go check them.*

project_story.md §15-16

A.5 Async-Reset DRC Violations: Chasing One Root Cause Through Twelve Files

Post-synthesis DRC reported 40+ warnings that block-RAM control logic was driven by asynchronously reset registers -- a pattern Xilinx's own documentation warns can corrupt memory contents outside what static timing analysis can see.

Investigation. What looked like a three-file fix (the buffer modules DRC actually named) grew to twelve files across three re-synthesis rounds: fixing the named files made the warning move to a different register one hierarchy level deeper each time -- first into a nested submodule, then into an unrelated width-adapter module with no "buffer" in its name at all. A project-wide grep for the actual anti-pattern (the specific asynchronous-reset sensitivity-list syntax) eventually enumerated the full remaining scope in one pass, rather than continuing to whack-a-mole one violation at a time.

Fix. Converted 60 `always` blocks across 12 files from asynchronous to synchronous reset -- Xilinx's own recommended remedy -- and re-verified bit-for-bit identical simulation results after every round.

Why this matters. *A DRC violation names the nearest offender on a signal path, not the full extent of the underlying pattern -- the same root cause can recur at every upstream hop in a handshake network, in modules that share no obvious naming relationship with where the symptom first appeared. This is also a clean example of when to stop iterating on individual violations and instead do one bounded, project-wide audit.*

project_story.md §18

A.6 Discovering the Full Pipeline Had Never Actually Been AXI-Wrapped

Before running full SoC integration, a check of the integration script's actual scope revealed it only ever wrapped the standalone matmul engine in AXI -- the complete matmul-Softmax-GELU pipeline this project had been calling "the accelerator" for many prior sections had never been given any AXI wrapping at all, only the raw ports the testbenches drive directly.

Investigation. Caught before running anything, by checking what the integration script actually targeted rather than assuming its scope matched the project's current vocabulary.

Fix. Built a new AXI4-Lite/AXI4-Stream wrapper for the full pipeline, deliberately keeping the underlying RTL's own parameter names instead of introducing new aliases -- a direct, explicit application of the lesson from A.4, since renaming was exactly what caused that earlier drift bug. The new wrapper needed zero configuration overrides in the integration script as a direct result.

Why this matters. *"Run the existing integration script" is not always as simple as running it -- checking that its scope still matches what the rest of the project means by "the accelerator" is worth a pause before spending a synthesis run on the wrong target. This is also a strong example of a documented lesson actively changing how the next piece of work was built, not just how a bug was fixed after the fact.*

project_story.md §19

A.7 A Synthesis Attribute That Silently Did Nothing

DSP48E1 hard-macro usage sat at just 5.91% while LUT usage climbed toward 58% at the full-SoC level -- an inversion of what a matmul-dominated systolic array accelerator should look like. The first attempt to force multiplications onto DSP48E1s, by placing a `use_dsp` synthesis attribute directly above the multiply-accumulate `always` block, produced byte-identical utilization on re-synthesis: no error, and no effect.

Investigation. Per Xilinx UG901, the `use_dsp` attribute must attach to the register declaration that holds the multiply result, not the enclosing procedural block -- an attribute on the `always` statement itself isn't a recognized attachment point and is silently dropped during elaboration, with no warning.

Fix. Changed the relevant output from `output reg` to `output wire`, introduced a per-branch internal register carrying the attribute correctly, and drove the port via a continuous assignment -- functionally and cycle-timing identical to the original RTL. Re-synthesis then showed DSP48E1 usage jump from 13 to 205 and CARRY4 usage drop by more than half. True dual-8-bit DSP packing was considered and explicitly ruled out first, since it would have required restructuring the systolic array's timing skew, a real microarchitecture change; the chosen fix was a lower-risk partial 1:1 DSP mapping (12 of 16 systolic rows) sized to fit the chip's DSP budget.

Why this matters. *A synthesis pragma that produces no error and no effect is exactly the kind of silent failure that's easy to miss if you only check for compile errors, not utilization numbers before and after. This is one of the project's strongest single findings*

and worth being able to explain precisely (attribute-attachment rules, wire-vs-reg, why the fix preserves identical behavior) in an interview.

project_story.md §20

A.8 Same Chip, Different Board: the PYNQ-Z1 to PYNQ-Z2 Hardware Audit

The physical board on hand is a PYNQ-Z2, but the whole flow had been built and verified against PYNQ-Z1 board files.

Investigation. Both boards use the identical Zynq-7020 chip, so everything living inside the fabric (RTL, timing closure, utilization, the complete absence of any user I/O constraints file) travels unchanged. The real risk was narrower and easy to overlook: the integration script's PS7 configuration hardcodes not just a cosmetic board-name string but the actual DDR3 board-preset values -- part number, per-pin trace-length and delay figures -- that model the PYNQ-Z1 PCB's specific DDR3 layout, which is a different physical board than the PYNQ-Z2 actually being used.

Fix. Deliberately did not hand-patch plausible-looking replacement numbers, since a fabricated-but-plausible DDR calibration value is worse than an honestly-wrong one -- it would look fixed while still being invented. Updated the board-part string to attempt the correct board identifier and left explicit inline comments flagging exactly which values are still PYNQ-Z1-specific, so the gap can't be silently forgotten. The correct fix (installing the board vendor's official Vivado board files and re-applying the preset from the GUI) was documented as the next concrete step rather than approximated.

Why this matters. *"Same silicon part number" and "same board" are not the same claim -- a good interview answer distinguishes precisely which parts of a hardware design travel with the chip and which are tied to the physical board it sits on, and explains why guessing a DDR calibration number is actively worse than leaving it visibly unresolved.*

project_story.md §22

A.9 A Comprehensively Stale Software Driver, Found and Rebuilt From the Real Register Map

The repository's existing PYNQ/Python driver (`apps/MM.py`) predated the project's later integration of the full matmul-Softmax-GELU pipeline. It configured the wrong array size, hardcoded peripheral addresses that didn't match the real integration script's address map, and only knew about the old 4-register matmul-only control interface.

Investigation. Rather than patch individual wrong values, the whole register/DMA contract was re-derived directly from the current RTL wrapper's own header comments and the real generated hardware description -- the same discipline established for the integration script in A.4. On the very first real hardware run, this caught a second, independent mismatch: PYNQ names a custom AXI-Lite peripheral after its actual AXI interface name, not its block instance name, so a hardcoded attribute path would have failed the moment it was used.

Fix. Rewrote the driver from scratch against the real address map and register layout, and replaced the hardcoded IP-attribute access with a small resolver helper that searches the overlay's actual IP dictionary instead of assuming one specific naming convention.

Why this matters. *A driver that looks complete (helper functions, shape validation, error handling) can still be built entirely against the wrong version of the hardware -- every one of its wrong values traced back to a single root cause, that nobody had updated it since the hardware it targeted changed. It's also a clean example of not trusting an assumption about how a tool names things until checking the tool's own actual output.*

project_story.md §23, §25

A.10 A Real, Reproducible Hardware/Software Mismatch, Diagnosed Methodically

The first numerical test on real PYNQ-Z2 hardware showed 11 of 256 output elements (about 4.3%) outside the golden model's tolerance -- every failing output was exactly zero, against expected values around 0.05-0.10.

Investigation. Investigated in a fixed order, ruling out cheaper explanations before touching RTL: re-ran the identical input three times and got bit-for-bit identical failures, ruling out timing noise; confirmed the FIFO-overflow safeguard hadn't fired; correlated 15 additional trials against each element's distance from its row's maximum value; and hand-traced the Softmax/Exp/Ln modules for a specific zero-output hypothesis, which the trace refuted. Rather than keep guessing by hand, wrote a standalone debug testbench reproducing the exact failing row through the Softmax module alone, with a full internal-signal trace. It came back clean: the isolated module computed the correct, non-zero answer given the same inputs -- proving the bug, if it was a logic bug at all, had to be somewhere the isolated test couldn't reach (the real data path feeding it, the actual register configuration, or something only real hardware timing could expose).

Fix. Not resolved by this test alone -- it correctly redirected the search rather than closing it (see A.11).

Why this matters. *A clean negative result is not a dead end, it's a boundary: once a module is proven correct in isolation, the remaining search space is provably everything outside it, not "somewhere in the module, just not the spots already checked." This entry is a strong interview answer to "tell me about a hard bug you debugged," since it shows a disciplined process (cheapest checks first, structural before circumstantial) rather than a lucky guess.*

project_story.md §26, §28

A.11 The Real Root Cause: a Tiny-Shape Edge Case, Not a Design Flaw

Following A.10's redirection, a targeted "marker row" hardware test (a unique spike per row, so a correctly-framed pipeline reproduces the spike's exact column position) showed the data-framing path itself was wrong even for a single row, and a sweep across small row counts hung outright at two rows.

Investigation. Every failing test up to this point -- including every test in A.10 -- used matrix-block-count values far smaller than anything the design had ever actually been verified at. Re-running the exact shape and configuration the full-pipeline simulation had already verified end-to-end (200×96×160) through the same real hardware path produced 0 of 32,000 output elements outside tolerance: the accelerator is correct on real silicon at the scale it was designed and verified for.

Fix. Reading the input-buffer's row-address counter logic directly (not hand-waved) found one concrete cause for the single-row failure: a wraparound condition that is mathematically correct for every row count except exactly one, where the wrap value it checks for can never actually occur. The two-row hang was traced to the same small-block-count territory but was not fully root-caused, and was explicitly left open rather than claimed fixed.

Why this matters. *This is the project's central verification lesson: a "quick" small smoke test accidentally exercised a completely different, never-validated region of the design's parameter space, and nearly turned a genuine but narrow edge case into a false alarm about the whole accelerator's correctness. The decisive move wasn't a cleverer hypothesis -- it was re-running the exact condition already proven correct and confirming the result actually matched. A strong interview point: distinguishing "the design is wrong" from "my test exercised an unverified corner of the design" is a real, valuable engineering skill.*

project_story.md §29

A.12 Refusing to Report a Speedup the Pipeline Doesn't Actually Support

Asked for a complete end-to-end Vision Transformer (ViT) benchmark: run a real ViT model, accelerate it with the hardware, and report the speedup.

Investigation. Before writing any benchmark code, checked whether the accelerator's fixed pipeline (matmul, then per-row Softmax, then GELU, fused with no way to tap the intermediate result) structurally matches any real ViT sub-block. It doesn't: self-attention needs matmul-softmax-matmul with no GELU in between, and the MLP block needs matmul-GELU-matmul with no Softmax in between. There is no ViT operation this specific fixed pipeline can correctly substitute into.

Fix. Presented this finding honestly rather than silently building a benchmark that would have reported a broken classification alongside a fabricated speedup number. The chosen alternative: a real, independently validated NumPy reimplementation of ViT-Tiny for a genuine software baseline and correctness check (matching the real PyTorch reference to float32 noise), plus a separate, clearly-labeled hardware kernel-throughput benchmark that never claims to represent an accelerated end-to-end ViT run.

Why this matters. *The most valuable check before writing a benchmark is often not the code, it's whether the comparison it will report is actually true. This is a strong example of scientific/engineering integrity under pressure to produce an impressive number -- exactly the kind of judgment call an interviewer may probe for directly.*

project_story.md §30

A.13 A 64KB DMA Ceiling, and Why the Obvious Workaround Would Have Silently Corrupted Data

The first real ViT-representative kernel benchmark run succeeded at one matrix shape but hard-failed at a larger one: the DMA engine refused the transfer outright because it exceeded a 65,536-byte maximum.

Investigation. Root cause: the three DMA cores were synthesized with a 16-bit transfer-length register, capping any single transfer at 64KB, and the larger shape's data exceeded that. The obvious-looking fix -- split the oversized transfer into several smaller DMA calls -- was checked against the actual RTL before being used: the input buffer's write-address counters reset to zero on the stream's end-of-frame signal, and this DMA mode asserts that signal at the end of every transfer, not only a genuine final one. Two chunked transfers would each look like "the last one" to the RTL, so the second chunk would silently overwrite the first from address zero instead of continuing it -- corrupting the buffer rather than raising any visible error.

Fix. Capped the affected benchmark shape at the largest size that fits within the 64KB ceiling, labeled explicitly (in code and in printed output) as DMA-limited rather than the true target dimension. The real fix -- a wider transfer-length register, or genuine Scatter-Gather DMA with explicit end-of-frame control per descriptor -- needs re-synthesis and was logged as a specific, well-understood open item rather than implemented under time pressure.

Why this matters. *The fastest-looking fix for a hardware error is not automatically the correct one -- tracing exactly what a workaround would do to the specific RTL logic involved caught a data-corrupting bug before it shipped. This is a good example of hardware/software co-design intuition: a software-only fix (chunking) can be actively unsafe when the hardware's own protocol semantics (what "last transfer" means at the RTL level) don't match what the software layer assumes.*

project_story.md §31

Appendix B: General Engineering Principles

The individual problems in Appendix A share a smaller number of recurring principles. They are collected here as a standalone summary -- useful as a direct answer to a broader interview question such as "what did you learn from this project," independent of any single bug.

- **Infrastructure first.** Verify infrastructure and environment assumptions before a deep technical investigation -- the single largest time cost early in this project (project_story.md §1-5) turned out not to be a design bug at all.
- **A fix isn't proven until the original symptom is confirmed gone.** Ideally more than once, and ideally by reproducing the exact original failing condition, not a similar-looking one.
- **`generate` loops don't scale as lookup tables.** One hardware object per array element is correct for genuinely parallel structure and catastrophic for a large data array; use an on-demand function instead.
- **Headroom and sizing parameters must scale in lockstep with data size.** A previously-fixed bug can silently reappear at a new scale if a related sizing parameter wasn't raised with it -- read and respect a file's own inline warnings about this.
- **A fix to a shared interface needs every consumer re-checked.** Not just the interface declaration itself -- a testbench or caller can keep silently exercising only the old, narrower behavior even after the interface is fixed.
- **Externally-adopted integration scripts are not authoritative just because they were machine-generated.** Audit their full parameter list against the verified design's own declarations, not only the one value already suspected wrong -- the values nobody explicitly overrode are typically the most dangerous ones, because nothing prompts a reviewer to go check them.
- **Simulation and post-synthesis design-rule checking catch different classes of bug.** A clean simulation proves functional correctness under the testbench's own stimulus and timing model, not silicon-level correctness -- asynchronous resets driving block-RAM control logic are a textbook example of something a behavioral simulator cannot expose but that a DRC pass exists specifically to catch.
- **A design-rule violation names the nearest offender, not the full extent of the pattern.** The same root cause can recur at every upstream hop in a signal path, in modules with no obvious naming relationship to where the symptom first appeared -- a project-wide audit for the actual anti-pattern is more reliable than iteratively chasing wherever the next violation happens to point.
- **A synthesis pragma that produces no error can still silently do nothing.** Attribute-attachment rules matter -- check the actual utilization numbers before and after, not just whether the build completed without complaint.
- **"Same chip" and "same board" are different claims.** Anything that lives purely in the silicon fabric travels between two boards sharing the same part number; anything that models the physical PCB (DRAM calibration, board-preset metadata) does not, and a plausible-looking guessed value is worse than an honestly-flagged gap.
- **A driver or script that looks complete can still target the wrong version of the hardware.** Re-derive an interface contract from the current design's own source of truth rather than trusting that an existing file was ever updated when the design changed underneath it.
- **A hardware bug search needs the same scale discipline as everything else.** Testing "quickly" with a small, convenient shape can accidentally exercise a completely different, never-validated region of the design's parameter space -- when in doubt, re-run the exact condition already proven correct and confirm the result actually matches, before concluding the design itself is at fault.
- **The most valuable check before generating a result is whether the comparison is even true.** An accelerator, benchmark, or workaround that looks plausible is not automatically valid for the specific case at hand -- checking this explicitly, in writing, before producing a number is worth the pause, especially for results that will be reported to someone else.